

Representación de la información en el sistema informático

Módulo: Sistemas informáticos

Contenido

1.	REPRESENTACIÓN DE LA INFORMACIÓN EN EL SISTEMA INFORMÁTICO.....	2
1.1	SISTEMAS DE NUMERACIÓN POSICIONALES (I).....	2
1.2	SISTEMAS DE NUMERACIÓN POSICIONALES (II)	3
1.2.1	Teorema Fundamenta de la Numeración	3
1.2.2	Sistema Decimal	4
1.2.3	Sistema Binario	5
1.2.4	Sistema Octal	5
1.2.5	Sistema Hexadecimal	6
1.3	CONVERSIÓN ENTRE SISTEMAS DE NUMERACIÓN (I).....	7
1.3.1	CONVERSIÓN DE CUALQUIER SISTEMA A DECIMAL	7
1.3.2	CONVERSIÓN DE DECIMAL A OTROS SISTEMAS DE NUMERACIÓN	7
1.3.3	Conversión decimal-binario.....	8
1.3.4	Conversión binario-decimal.....	8
1.4	CONVERSIÓN ENTRE SISTEMAS DE NUMERACIÓN (II)	9
1.4.1	Conversión decimal-octal	9
1.4.2	Conversión octal-decimal	9
1.4.3	Conversión decimal-hexadecimal.....	9
1.4.4	Conversión hexadecimal-decimal.....	9
1.5	REPRESENTACIÓN DE NÚMEROS EN LOS SISTEMAS INFORMÁTICOS: NÚMEROS ENTEROS.....	10
1.5.1	Signo y magnitud.....	10
1.5.2	Complemento a 1.....	11
1.5.3	Complemento a 2.....	11
1.6	REPRESENTACIÓN DE NÚMEROS EN LOS SISTEMAS INFORMÁTICOS: NÚMEROS REALES.....	12
1.6.1	Representación en coma flotante	12
1.7	SISTEMAS DE REPRESENTACIÓN: CÓDIGOS ALFANUMÉRICOS (I).....	14
1.7.1	Código ASCII	14
1.7.2	Código EBCDIC.....	15
1.7.3	Código UNICODE	15

1. REPRESENTACIÓN DE LA INFORMACIÓN EN EL SISTEMA INFORMÁTICO

Caso práctico:



¿Cuáles son los mecanismos utilizados por el ordenador para representar los datos?

La información que nosotros manejamos en nuestra vida cotidiana, está expresada principalmente en forma de textos, números o imágenes fijas o imágenes en movimiento.

Para que los ordenadores puedan procesar esta información, necesitamos convertirla o traducirla de alguna forma al lenguaje utilizado por las máquinas. Los sistemas

informáticos actuales, están basadas en el código binario, y únicamente son capaces de diferenciar entre dos estados: 1 y 0. Por tanto necesitamos un código o sistema establecido para traducir la información que nosotros manejamos habitualmente al lenguaje utilizado por los sistemas informáticos. A continuación vamos a ver cómo se representan los números en los sistemas informáticos, así como algunos sistemas de codificación.

1.1 SISTEMAS DE NUMERACIÓN POSICIONALES (I)

Destacado:



El hombre, desde el principio de los tiempos, intentó buscar un sistema para contabilizar. Puesto que disponemos de dos manos con cinco dedos cada una, éstos nos permiten representar 10 símbolos, o números. De aquí surgió el primer sistema de numeración: el decimal.

Un código o sistema de numeración es un conjunto de símbolos y reglas que se utilizan para representar cantidades.

En todos los sistemas de numeración existe un elemento que caracteriza al propio sistema y se le da el nombre de **base** del sistema de numeración. La base del sistema de numeración, es el número de símbolos distintos que se utilizan para poder representar la información en ese sistema determinado, por ejemplo el sistema decimal tiene base 10,

es decir utiliza 10 símbolos distintos (del 0 al 9), el sistema binario tiene base 2 y utiliza 2 símbolos (0 y 1).

Otro concepto que debemos conocer cuando hablamos de un sistema de numeración es el **rango de representación**, que es el conjunto de cantidades posibles que podemos representar dado un número de cifras determinado (n).

El rango de representación se determina elevando la base del sistema de representación al número de cifras que se vayan a utilizar en la codificación.

Por ejemplo, en un sistema de representación de 4 cifras (posiciones) en base 2, su rango de representación es: $2^4 = 16$, es decir, 16 es el número de combinaciones distintas que podemos hacer con 4 posiciones y dos símbolos distintos de representación.

Este tipo de sistema de numeración recibe el nombre de **sistema de numeración posicional**.

Sistema de numeración posicional: Es aquel que al representar una cantidad mediante una cadena de símbolos, el significado de cada uno de los símbolos que la forman varían en función de la posición que ocupen dentro de la cadena.

Por ejemplo: si tenemos los números 84 y 48, vemos que el 84 tiene una cadena de símbolos el 8 y el 4, mientras que el 48 tiene una cadena de símbolos compuesta por el 4 y el 8. Cada uno de esos símbolos tiene un valor distinto dependiendo de su posición dentro de la cadena que forma el número.

En el 84 el 4 representa las unidades y el 8 las decenas. En el 48 el 8 representa las unidades y el 4 las decenas. Lo cual significa que en función de la posición que ocupe el número dentro de la cadena, su valor varía.

1.2 SISTEMAS DE NUMERACIÓN POSICIONALES (II)

A continuación vamos a ver algunos sistemas de numeración posicionales.

1.2.1 Teorema Fundamental de la Numeración

Este teorema establece la forma general de construir números en un sistema de numeración posicional. Primero estableceremos unas definiciones básicas:

N , número válido en el sistema de numeración.

b , base del sistema de numeración. Número de símbolos permitidos en el sistema.

d_i , un símbolo cualquiera de los permitidos en el sistema de numeración.

n , número de dígitos de la parte entera.

, coma fraccionaria. Símbolo utilizado para separar la parte entera de un número de su parte fraccionaria.

k , número de dígitos de la parte decimal.

La fórmula general para construir un número N , con un número finito de decimales, en un sistema de numeración posicional de base b es la siguiente:

$$N = \begin{cases} \langle d_{(n-1)} \dots d_1 d_0, d_{-1} \dots d_{-k} \rangle = \sum_{i=-k}^n d_i b^i \\ N = d_n b^n + \dots + d_1 b^1 + d_0 b^0 + d_{-1} b^{-1} + \dots + d_{-k} b^{-k} \end{cases}$$

El valor total del número será la suma de cada dígito multiplicado por la potencia de la base correspondiente a la posición que ocupa en el número.

Esta representación posibilita la realización de sencillos algoritmos para la ejecución de operaciones aritméticas.

Ejemplo en el sistema decimal

En el sistema decimal los símbolos válidos para construir números son $\{0,1,\dots,9\}$ (0 hasta 9, ambos incluidos), por tanto la base (el número de símbolos válidos en el sistema) es diez

En la figura inferior podemos ver el teorema fundamental de la numeración aplicado al sistema decimal.

$$\begin{aligned} N &= d_n \dots d_1 d_0, d_{-1} \dots d_{-k} = \\ d_n \cdot 10^n + \dots + d_1 \cdot 10^1 + d_0 \cdot 10^0 + d_{-1} \cdot 10^{-1} + \dots + d_{-k} \cdot 10^{-k} &= \\ N &= \sum_{i=-k}^n d_i \cdot 10^i \end{aligned}$$

$$1492,36 = 1 \cdot 10^3 + 4 \cdot 10^2 + 9 \cdot 10^1 + 2 \cdot 10^0 + 3 \cdot 10^{-1} + 6 \cdot 10^{-2}$$

1.2.2 Sistema Decimal

Es el que entendemos y utilizamos todos los humanos de forma habitual, es un sistema de numeración en base 10. Utiliza 10 símbolos (del 0 al 9) para representar cualquier cantidad.

Su rango de representación será: 10^n , donde n es el número de cifras o posiciones que se vayan a utilizar.

Este sistema de numeración es **posicional**, el dígito más a la derecha representa las unidades y queda multiplicado por 1 (para las unidades sería 10^0), el siguiente dígito representa a las decenas quedando multiplicado por 10 (10^1) así sucesivamente.

EJEMPLO: Descomposición del número decimal 433.

$$4 \times 10^2 + 3 \times 10^1 + 3 \times 10^0 = 400 + 30 + 3 = 433. \text{ (Teorema fundamental de numeración)}$$

Por ser el sistema que conocemos todos será el utilizado para conocer cualquier cantidad representada en otro sistema de numeración.

1.2.3 Sistema Binario

0	1	10	11	100	101	110	111	1000	1001	1010
0	1	2	3	4	5	6	7	8	9	10
1011	1100	1101	1110	1111	10000	10001	10010	10011	10100	
11	12	13	14	15	16	17	18	19	20	

Sistema de numeración binario.

El ordenador utiliza internamente este sistema de numeración. Es un sistema de numeración en base 2. Utiliza únicamente 2 símbolos (el 0 y el 1) para representar cualquier cantidad. Cada uno de los dígitos que componen el número representado en este sistema se le denomina **Binary digit**.(bit)

El valor posicional de un dígito dentro de un número binario se basa en la progresión de potencia de 2.

EJEMPLO: La representación del número decimal 13 en binario es 1101, siendo su descomposición la siguiente (Teorema fundamental de la numeración):

$$13 \text{ (base10)} = 1101 \text{ (base 2)} = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 8 + 4 + 0 + 1 = 13$$

1.2.4 Sistema Octal

0	1	2	3	4	5	6	7	10	11	12
0	1	2	3	4	5	6	7	8	9	10
13	14	15	16	17	20	21	22	23	24	
11	12	13	14	15	16	17	18	19	20	

Sistema de numeración octal.

Es un sistema de numeración en base 8 que utiliza 8 símbolos (del 0 al 7) para representar cualquier cantidad.

El valor posicional de un dígito dentro de un número en base octal se basa en la progresión de potencia de 8.

EJEMPLO: La representación del número decimal 78 en octal es 116, siendo su descomposición la siguiente:

$$1 \times 8^2 + 1 \times 8^1 + 6 \times 8^0 = 64 + 8 + 6 = 78.$$

1.2.5 Sistema Hexadecimal

0	1	2	3	4	5	6	7	8	9	
0	1	2	3	4	5	6	7	8	9	
A	B	C	D	E	F	10	11	12	13	14
10	11	12	13	14	15	16	17	18	19	20

Sistema de numeración hexadecimal.

Es un sistema de numeración en base 16 y utiliza 16 símbolos (del 0 al 9 y las letras A,B,C,D,E,F) para representar cualquier cantidad. Cada una de las letras representa un valor, A=10; B=11; C=12; D=13; E=14; F=15.

El valor posicional de un dígito dentro de un número en base hexadecimal se basa en la progresión de potencia de 16.

EJEMPLO: La representación del número decimal 78 en hexadecimal es 4E, siendo su descomposición la siguiente:

$$4 \times 16^1 + 14 \times 16^0 = 64 + 14 = 78.$$

AUTOEVALUACIÓN

¿Cuál de los siguientes números NO representa una cifra en binario?

- a) 1100100010101
- b) 10010010
- c) 102001010001 (Correcto)
- d) 0

¿Cuál de los siguientes números NO representa una cifra en octal?

- a) 187475746 (Correcto)
- b) 664771627
- c) 1100100101
- d) 0

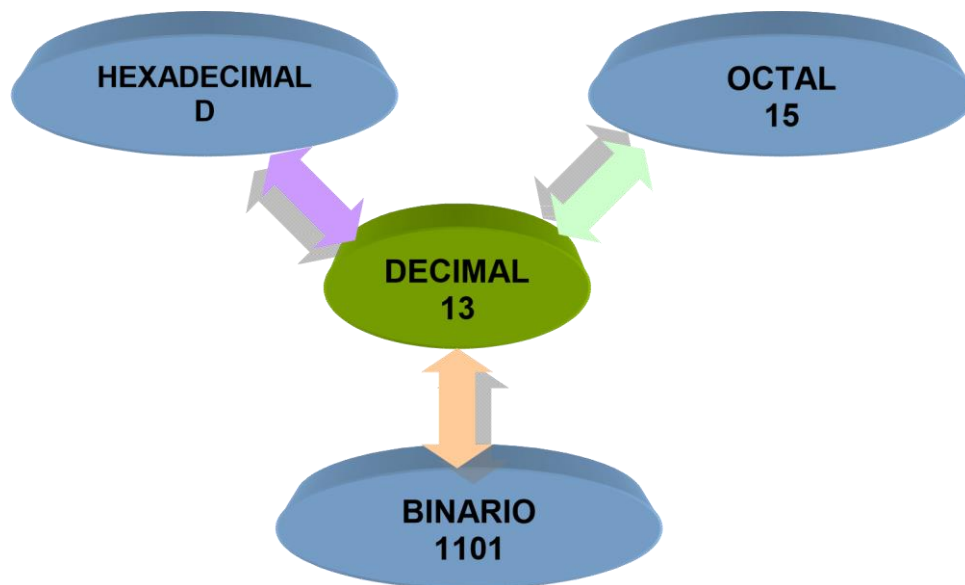
¿Cuál de los siguientes números NO representa una cifra en decimal?

- a) 1100100010101
- b) 10010010
- c) 102001B010001 (Correcto)
- d) 0

¿Cuál de los siguientes números NO representa una cifra en hexadecimal?

- a) 100101010100
- b) 13840AB183H0 (Correcto)
- c) 102001B010001
- d) 0

1.3 CONVERSIÓN ENTRE SISTEMAS DE NUMERACIÓN (I)



A continuación vamos a ver cómo cambiar de un sistema de numeración a otro sistema que utilice diferente número de símbolos (base).

1.3.1 CONVERSIÓN DE CUALQUIER SISTEMA A DECIMAL

Consiste en transformar una cantidad dada o expresada en un sistema de numeración concreto en otra cantidad expresada en el sistema decimal y que ambas sean equivalentes.

Para la conversión de un número en cualquier base a decimal se aplicará el Teorema Fundamental de la Numeración que relaciona una cantidad expresada en cualquier sistema de numeración con su equivalente en base 10.

Teorema fundamental de la numeración:

Dado un número de n cifras con las cifras $X_n, \dots, X_2, X_1, X_0$ que está en base B , su valor decimal equivalente viene representado por la fórmula:

$$X_n \times B^n + \dots + X_2 \times B^2 + X_1 \times B^1 + X_0 \times B^0$$

Ejemplo: Convertir el número 56 que está expresado en octal (base 8), a decimal.(base 10)

$$6 \times 8^1 + 5 \times 8^0 = 6 + 40 = 46 \text{ (decimal)}$$

1.3.2 CONVERSIÓN DE DECIMAL A OTROS SISTEMAS DE NUMERACIÓN

El procedimiento general para pasar de decimal a cualquier sistema de numeración, consiste en:

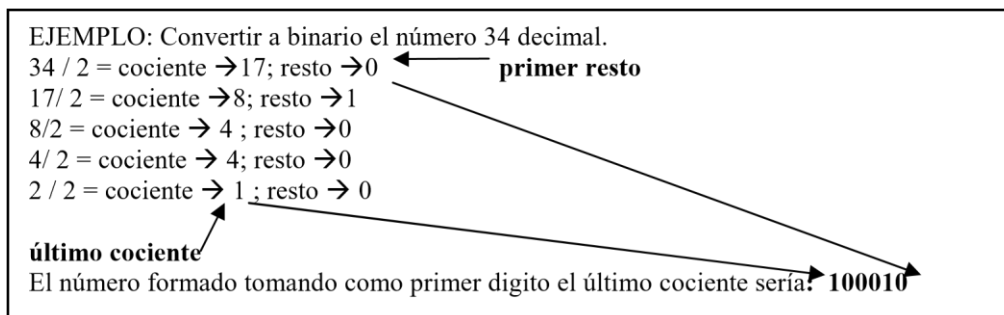
- Realizar divisiones sucesivas del número en decimal entre la base del sistema de numeración a la que queramos cambiar (si es a binario entre 2, si es a octal entre 8, si es a hexadecimal entre 16, ...), hasta que el cociente de la división sea menor que la base del sistema de numeración al que queremos convertir el número (si es a binario, hasta que el cociente sea menor que 2, si es a octal hasta que el cociente sea menor que 8, si es a hexadecimal, hasta que el cociente sea menor que 16.....)
- El número convertido a la base deseada, se forma tomando el último cociente, que será la cifra más a la izquierda, y los restos sucesivos, empezando por el último, de forma que la cifra más a la derecha del número será el primero de los restos.

Vamos a ilustrarlo con ejemplos de conversión entre los diferentes sistemas:

1.3.3 Conversión decimal-binario

Se realiza mediante divisiones sucesivas entre 2. Los pasos a seguir son:

1. Se divide el número entre 2 sucesivamente.
2. Los sucesivos cocientes se siguen dividiendo entre 2 hasta que el cociente sea menor que dos.
3. El número se formará cogiendo el último cociente y los distintos restos de derecha a izquierda, siendo el primer dígito (el dígito más a la izquierda) del número formado, el último cociente y el último dígito el primer resto.



1.3.4 Conversión binario-decimal

Se realiza aplicando el teorema fundamental de la numeración.

EJEMPLO: Convertir a decimal el número 100010 binario.

Teorema fundamental de la numeración $X_n \times B^n + \dots + X_2 \times B^2 + X_1 \times B^1 + X_0 \times B^0$ donde $B=2$ (binario) y $X_0=0, X_1=1, X_2=0, X_3=0, X_4=0, X_5=1$, que son las cifras del número. Aplicándolo obtenemos:

$$0 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 0 \times 2^3 + 0 \times 2^4 + 1 \times 2^5 = 0 + 2 + 0 + 0 + 0 + 32 = 34.$$

Ejercicio resuelto:

Convertir el número hexadecimal 10A a decimal, utilizando el teorema fundamental de la numeración:

Solución: $10A = 10 (A) \times 16^0 + 0 \times 16^1 + 1 \times 16^2 = 10 + 0 + 256 = 266.$

Ejercicio resuelto.

Convertir el número decimal 22 a binario:

Solución: $22/2=11(\text{resto } 0)$; $11/2=5(\text{resto } 1)$; $5/2=2(\text{resto } 1)$; $2/2 = 1(\text{resto } 0)$. Solución: 10110.

1.4 CONVERSIÓN ENTRE SISTEMAS DE NUMERACIÓN (II)

1.4.1 Conversión decimal-octal

Se realiza mediante divisiones sucesivas entre 8. Los pasos a seguir son:

1. Se divide el número dado entre 8.
2. Los sucesivos cocientes se siguen dividiendo entre 8 hasta que el cociente sea menor que 8.
3. El número se formará cogiendo el último cociente y los distintos restos de derecha a izquierda, Siendo el primer dígito del número formado el último cociente y el último dígito del número formado el primer resto. (El procedimiento es el mismo que para pasar de decimal a binario) EJEMPLO: Pasar a octal el número 32 decimal.

$32/8 = \text{cociente } 4; \text{ resto } 0$

El número formado es el 40.

1.4.2 Conversión octal-decimal

Se realiza aplicando el teorema fundamental de la numeración:

EJEMPLO: Pasar a decimal el número 40 octal

$$0 \times 8^0 + 4 \times 8^1 = 0 + 32 = 32.$$

1.4.3 Conversión decimal-hexadecimal

Se realiza mediante divisiones sucesivas entre 16. Los pasos a seguir son exactamente iguales que para la conversión a binario y octal.

EJEMPLO: Pasar a hexadecimal el número 31 decimal.

$31/16 = \text{cociente } 1; \text{ resto } 15$ (Recordamos que 15 es F en hexadecimal) El número formado es el 1F.

1.4.4 Conversión hexadecimal-decimal

Se realiza aplicando el teorema fundamental de la numeración:

EJEMPLO: Pasar a decimal el número 1F hexadecimal. (Recordamos que 15 es F en hexadecimal) $15 \times 16^0 + 1 \times 16^1 = 15 + 16 = 31$.

AUTOEVALUACIÓN

1. Pasa a base 2 el número 90, que está en base 10.

RESPUESTA: 1011010

2. Pasa el número 1101 de base 2 a base 10.

RESPUESTA: 13

3. Convierte a hexadecimal el número decimal 41565

RESPUESTA: A25D

4. Expresa en decimal el número hexadecimal F03

RESPUESTA: 3843

5.- Convierte a hexadecimal el número decimal 28

RESPUESTA: 1C

6.- Convertir el número 132 de base 8 a base 16

RESPUESTA: 5A

1.5 REPRESENTACIÓN DE NÚMEROS EN LOS SISTEMAS INFORMÁTICOS: NÚMEROS ENTEROS



Un bit es un dígito binario (base 2), que es la mínima cantidad de información representable. En los sistemas digitales se asigna un número fijo n de bits para representar un número, donde n es la longitud de una palabra (conjunto de n bits). Una palabra está compuesta por un número de bits que pueden ser tratados en su conjunto o simultáneamente en una operación del sistema digital, siendo sus longitudes más corrientes: 8, 16 ó 32 bits, según el sistema.

Con n bits se pueden representar 2^n combinaciones distintas y por lo tanto 2^n números diferentes, por lo que existirán dos valores extremos, un máximo y un mínimo, que acotarán a todos los números representables.

1.5.1 Signo y magnitud

En este sistema de representación, se utiliza 1 bit para representar el signo, y el resto, en binario natural, para representar el número.

Por tanto utilizando este sistema, el rango de representación será:

Utilizando n bits $\Rightarrow -(2^{n-1}-1) \leq X \leq (2^{n-1}-1)$

Lo podemos ver mejor ilustrado con un ejemplo:

Si utilizamos 8 bits: $n=8$ bits, el rango de números que podemos representar utilizando este sistema será: $-(2^{8-1}-1) \leq X \leq (2^{8-1}-1)$, $-127 \leq X \leq 127$.

Puesto que 1 bit lo utilizamos para representar el signo, y los 7 restantes para representar los números.

Es decir, podemos representar los números enteros comprendidos entre el -127 y el 127.

A continuación vamos a codificar utilizando este sistema, los números 20 y -20: (utilizando 8 bits):

$$+20_{(10)} = \mathbf{0}0010100 \quad -20_{(10)} = \mathbf{1}0010100$$

(El bit en negrita representa el signo del número: 0 para números positivos y 1 para números negativos)

1.5.2 Complemento a 1

Al igual que en el sistema anterior, en este sistema se utiliza el primer bit para codificar el signo del número (0 para números positivos / 1 para números negativos).

El número se representa en binario si es positivo, y en complemento a 1 si es negativo.

El complemento a 1, de un número binario se obtiene intercambiando los ceros por unos y los unos por ceros.

EJEMPLO: Representación de los números 20 y -20 en complemento a 1, utilizando 8 bits:

$$+20_{(10)} = \mathbf{0}0010100_{(C1)}$$

$$-20_{(10)} = \mathbf{1}1101011_{(C1)}$$

(El bit en negrita representa el signo del número)

1.5.3 Complemento a 2

Primer bit para signo, (0 positivo/1 negativo). El número positivo se representa en binario.

Si es negativo:

1. se complementa a 1 (incluso signo)
2. se le suma 1 en binario, (sin acarreo final)

EJEMPLO: Representación de los números 68 y -68 en complemento a 2, utilizando 8 bits:

$$+68_{(10)} = 01000100_{(C2)}$$

$$-68_{(10)} \Rightarrow \quad 68: \mathbf{0}1000100$$

$$\quad \quad \quad C1: 10111011$$

$$\quad \quad \quad +1: \quad \quad \quad 1$$

$$\mathbf{1}0111100_{(C2)}$$

(El bit en negrita representa el signo del número: 0 positivo, 1 negativo)

Ejercicio resuelto:

Representa el número $-45_{(10)}$ en los sistemas de representación: signo y magnitud, complemento a 1, complemento a 2, y notación sesgada. Se utilizarán 8 bits para representarlo.

Solución:

Signo-magnitud: 10101101.

Complemento a 1: 11010010.

Complemento a 2: 11010011.

1.6 REPRESENTACIÓN DE NÚMEROS EN LOS SISTEMAS INFORMÁTICOS: NÚMEROS REALES

1.6.1 Representación en coma flotante

-1,3
1,05
-9,432
-10,23
3,1416
7,45
6,7682
-5,212

Esta representación se utiliza para representar números reales y enteros con un rango de representación mayor que el que ofrecen otros sistemas de representación. Con eso se consigue que el ordenador pueda tratar números muy grandes o muy pequeños.

En este sistema de representación en primer lugar hay que pasar el número a notación exponencial, lo que se denomina normalización.

La idea es descomponer el número en dos partes:

- Una **mantisa** (también llamada coeficiente o significando) que contiene los dígitos del número. Mantisas negativas representan números negativos.
- Un **exponente** que indica dónde se coloca el punto decimal (o binario) en relación al inicio de la mantisa. Exponentes negativos representan números menores que uno.

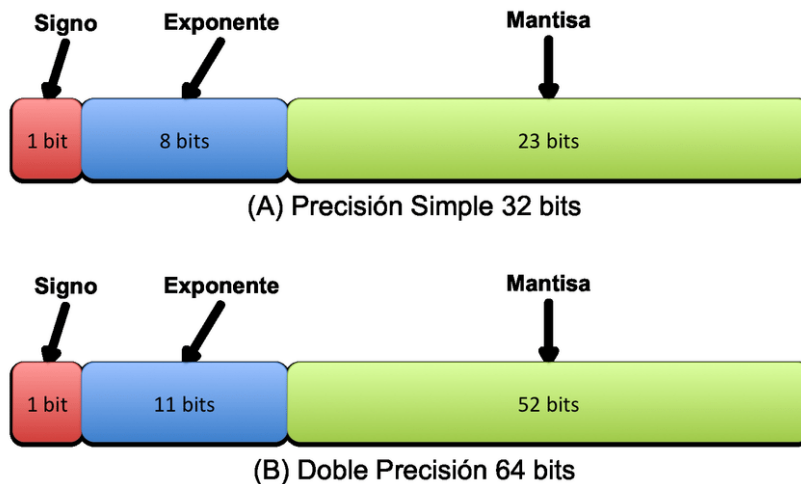
Mantisa	Exponente	Notación científica	Valor en punto fijo
1.5	4	$1.5 \cdot 10^4$	15000
-2.001	2	$-2.001 \cdot 10^2$	-200.1
5	-3	$5 \cdot 10^{-3}$	0.005
6.667	-11	$6.667e^{-11}$	0.0000000000667

Este formato cumple todos los requisitos:

- Puede representar números de órdenes de magnitud enormemente dispares (limitado por la longitud del exponente).
- Proporciona la misma precisión relativa para todos los órdenes (limitado por la longitud de la mantisa).

- Permite cálculos entre magnitudes: multiplicar un número muy grande y uno muy pequeño conserva la precisión de ambos en el resultado.

Casi todo el hardware y lenguajes de programación utilizan números de punto flotante en los mismos formatos binarios, que están definidos en el estándar [IEEE 754](#). Los formatos más comunes son de 32 o 64 bits de longitud total:



Hay algunas peculiaridades:

- La *secuencia de bits* es primero el bit del signo, seguido del exponente y finalmente los bits significativos.
- El exponente no tiene signo; en su lugar se le resta un **desplazamiento** (127 para sencilla y 1023 para doble precisión). Esto, junto con la secuencia de bits, permite que los números de punto flotante se puedan comparar y ordenar correctamente incluso cuando se interpretan como enteros.
- Se asume que el bit más significativo de la mantisa es 1 y se omite, excepto para casos especiales.
- Hay valores diferentes para **cero positivo y cero negativo**. Estos difieren en el bit del signo, mientras que todos los demás son 0. Deben ser considerados iguales aunque sus secuencias de bits sean diferentes.
- Hay valores especiales **no numéricos** (NaN, «not a number» en inglés) en los que el exponente es todo unos y la mantisa *no* es todo ceros. Estos valores representan el resultado de algunas operaciones indefinidas (como multiplicar 0 por infinito, operaciones que involucren NaN, o casos específicos). Incluso valores NaN con idéntica secuencia de bits *no* deben ser considerados iguales.

El denominado estándar **IEEE 754**, define la norma de representación de números en coma flotante más extendida por los computadores actuales.

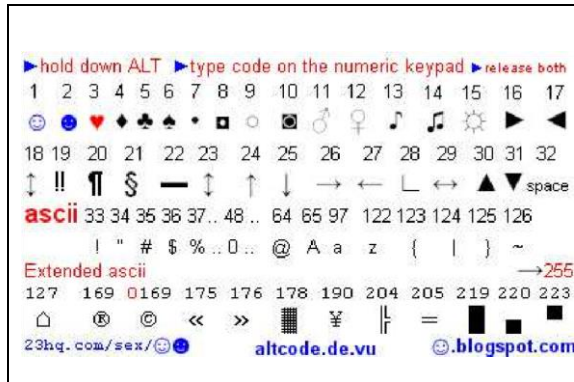
PARA SABER MÁS...

Pulsa en el enlace para saber más sobre la norma IEEE 754:

[Norma IEEE754](#)

http://es.wikipedia.org/wiki/IEEE_coma_flotante

1.7 SISTEMAS DE REPRESENTACIÓN: CÓDIGOS ALFANUMÉRICOS (I)



Hasta ahora hemos visto cómo se representan los diferentes tipos de números (números reales y números enteros) en los sistemas informáticos. Sin embargo, en los sistemas informáticos, no solamente se procesa información numérica (números) sino que se procesa también información alfanumérica (caracteres), como por ejemplo los caracteres del alfabeto, para poder introducir textos en un equipo. Para

representar estos caracteres (letras y signos del alfabeto), se utilizan otros sistemas de codificación: los llamados códigos alfanuméricos, que nos permiten codificar por ejemplo todas las letras del alfabeto. Entre estos códigos alfanuméricos los más conocidos son: el código ASCII y el código EBCDIC).

1.7.1 Código ASCII

El código ASCII (American Standard Code for Information Interchange) (Código normalizado americano para el intercambio de información) se utiliza para la representación de la información en los ordenadores aunque fue ideado para la transmisión de datos.

El código ASCII incluye 256 códigos divididos en dos conjuntos, estándar y extendido, de 128 cada uno. La unión de estos conjuntos permite representar todas las combinaciones posibles de 7 u 8 bits. El conjunto ASCII estándar utiliza 7 bits para cada código dando como resultado ($2^7=128$) códigos de caracteres (del 0 hasta 127) y el conjunto de ASCII extendido utiliza 8 bits para cada código, dando como resultado otros 128 códigos adicionales (del 128 al 255).

La distribución de los caracteres es el código ASCII es la siguiente:

- 32 caracteres de control.
- 64 caracteres que representan las letras mayúsculas y cifras.
- 32 caracteres para representar las letras minúsculas y algunos signos especiales.

Si quieres escribir cualquiera de los caracteres alfanuméricos incluidos entre el número 33 y el 255, sólo tienes que abrir el procesador de textos y activar el teclado numérico. Si ese teclado no se encuentra activado, sólo tienes que oprimir la tecla “Bloq Num” en el propio teclado (cuando está activado se reconoce porque se enciende el primer LED, situado encima de esa tecla, que aparece con el nombre “N/Lock”). Seguidamente se oprime la tecla “Alt” y se teclea, simultáneamente, sin soltarla, el número decimal correspondiente a la letra, número o signo del Código ASCII que queremos obtener. A continuación soltamos la tecla “Alt” y el carácter aparecerá escrito en el procesador.

Caracteres imprimibles del código ASCII Estándar (7 bits – 128 caracteres)											
Dec	Car.	Dec	Car.	Dec	Car.	Dec	Car.	Dec	Car.	Dec	Car.
32	Espacio	48	0	64	@	80	P	96	ˆ	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	\$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

1.7.2 Código EBCDIC

El código EBCDIC (Extended Binary Coded Decimal Interchange Code) (Código ampliado de intercambio decimal codificado en binario) fue el primero que se ideó como código interno de ordenadores. Fue un esquema de codificación desarrollado por IBM para utilizarlo en sus ordenadores como método de normalizado de asignación de valores binarios a los caracteres alfabéticos, numéricos de puntuación y de control de transmisión. El EBCDIC es igual que el esquema de codificación ASCII estándar, se diferencia por utilizar 8 bits para la codificación, lo que permite 256 caracteres posibles frente a los 128 del ASCII estándar.

Aunque este código no es muy utilizado en las microcomputadoras, sí es conocido y aceptado internacionalmente, siendo usado sobre todo como código de IBM para los mainframes (grandes ordenadores) y minicomputadoras.

1.7.3 Código UNICODE

Éste es un código de 16 bits y permite 65.536 representaciones posibles de caracteres. Fue pensado para paliar las deficiencias de los anteriores códigos cuando se utilizan alfabetos propios de lenguas autóctonas, de diferentes países e incluso de uso local, también dispone de un conjunto de caracteres matemáticos.

Los 256 caracteres primeros coinciden con los de cualquier código de 8 bits para evitar problemas de incompatibilidad entre diferentes códigos.

1.7.3.1 PARA SABER MÁS...

Pulsando en los enlaces tienes información ampliada sobre estos dos códigos. [Código EBCDIC](http://es.wikipedia.org/wiki/EBCDIC)
<http://es.wikipedia.org/wiki/EBCDIC>
[Código Unicode](http://es.wikipedia.org/wiki/Unicode)
<http://es.wikipedia.org/wiki/Unicode>